Continuous Feature Representations for FT-Transformers in Tabular Data

Yonatan Seleznev
seleznev@post.bgu.ac.il

Shiri K. Nitzan-Tzahor
shiricat@post.bgu.ac.il

Abstract

Deep learning models based on the Transformer architecture have recently been adapted for tabular data, yet their performance often depends strongly on how numerical features are represented. In this work, we investigate how different numerical feature representations affect the performance of the FT-Transformer on tabular prediction tasks. In addition to commonly used approaches, we evaluate two simple transformations applied prior to the embedding layer: standard Z-score normalization and an additional rank-based representation that we propose. While Z-score normalization is a simple and widely used preprocessing technique, the rank-based transformation maps each observation to its empirical percentile within the training distribution, preserving the ordering of values while removing dependence on absolute scale.

We compare four representations applied before the model's feature-specific linear projection: raw scalar inputs, Z-score normalization, the proposed rank-based representation, and Piecewise Linear Encoding (PLE). Experiments are conducted across five Kaggle/UCI tabular datasets with varying sizes and feature compositions, and an additional medical dataset provided by Sheba Medical Center. Model performance is evaluated using the ROC-AUC metric, and we also report runtime comparisons. We further compare the FT-Transformer results to a Random Forest baseline and perform a statistical comparison across datasets using the Friedman test. Although our proposed transformations did not outperform the alternative representations, the results indicate that the representation of numerical features can influence predictive performance. Among the evaluated methods, PLE often produced the strongest FT-Transformer results, although no method significantly outperformed the others across datasets. The experiments also show that Random Forest models achieve competitive predictive performance while requiring substantially less training time. These findings highlight the importance of numerical feature representation in transformer-based tabular models and motivate further exploration of simple distribution-aware transformations.

1 Introduction

Tabular data is one of the most common formats encountered in real-world machine learning applications. It consists of samples (rows) described by a fixed set of features (columns) and is widely used in domains such as healthcare, finance, manufacturing, and marketing. Despite the rapid success of deep learning (DL) in fields such as computer vision and natural language processing, traditional machine learning methods, particularly gradient-boosted decision trees (GBDT), often remain the strongest performers for tabular data [1, 2].

Several characteristics of tabular data make it challenging for deep learning models. Tabular datasets are often relatively small, increasing the risk of overfitting in complex neural architectures. In addition, tabular data lacks the strong spatial or sequential structure present in images or text, and

BGU IEM Deep Learning Course 2026 - Final Project.

frequently contains different feature types, missing values, and complex feature interactions. For these reasons, tree-based models such as XGBoost, Random Forest, and CatBoost continue to be widely used for tabular learning tasks [1, 3].

Motivated by the success of Transformers in other domains, recent work has explored attention-based architectures for tabular data [4]. One notable approach is TabTransformer [5], which applies Transformer layers to contextualize categorical feature embeddings before combining them with numerical features for prediction. Building on this idea, the Feature-Tokenizer Transformer (FT-Transformer) treats each feature as a token and processes them through Transformer layers, enabling the modeling of interactions between different features [4]. Although these architectures have shown competitive performance, they did not consistently outperform traditional tree-based methods across tabular tasks.

In this work, we investigate different representations of numerical features within the FT-Transformer framework. Specifically, we compare several numerical embedding strategies and analyze their effect on predictive performance across multiple tabular datasets.

2 Background

2.1 TabTransformer

The TabTransformer architecture consists of a column embedding layer, a stack of N Transformer layers, and a multilayer perceptron. Each categorical feature is embedded into a vector of dimension D, producing a sequence of embeddings that is processed by the Transformer layers to model interactions between features.

Let (x, y) denote a feature–target pair, where $x = \{x_{cat}, x_{cont}\}$ contains categorical and continuous features. Each categorical feature x_i with d_i categories is mapped to a learned embedding through an embedding lookup table e_{ϕ_i}. The resulting embeddings are passed through N Transformer layers, producing contextual representations $\{h_1, \ldots, h_m\}$.

These contextual embeddings are concatenated with the numerical features x_{cont} and passed to an MLP that outputs the final prediction $\hat{y}$. The model architecture is presented in Appendix A.1.

2.2 FT-Transformer

The FT-Transformer (Feature Tokenizer Transformer) addresses a key limitation of the Tab-Transformer: the asymmetric treatment of categorical and continuous features. While the Tab-Transformer applies the Transformer only to categorical embeddings and passes continuous features directly to the MLP, the FT-Transformer tokenizes all features into embeddings, allowing the Transformer to model interactions across the entire feature space. The architecture of FT-Transformer is shown in Figure 1.

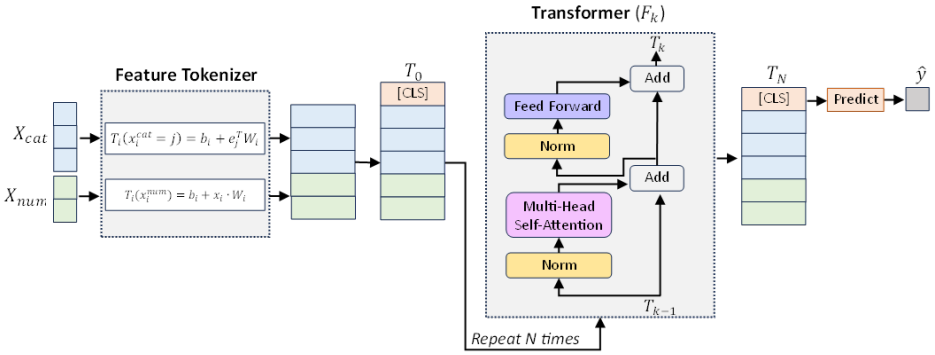

Figure 1: Architecture of the FT-Transformer.

We denote m and c as the number of categorical and continuous features respectively, D is the embedding dimension, and d_i is the number of classes for categorical feature i. We also define

$n = m + c$ as the total number of features.
The architecture consists of two components: a Feature Tokenizer, which maps all features to a sequence of embeddings $T \in \mathbb{R}^{n \times D}$, and a standard Transformer with a final linear prediction head applied to a special [CLS] token.
The Feature Tokenizer computes $T_i \in \mathbb{R}^D$ differently depending on the feature type. For categorical variables, T_i is defined:

$$T_i = b_i + e_j^T W_i \qquad \in \mathbb{R}^D$$

where $b_i \in \mathbb{R}^D$ is the learned bias for feature i, $W_i \in \mathbb{R}^{d_i \times D}$ is the learned embedding matrix, and $e_j \in \mathbb{R}^{d_i}$ is the one-hot encoding of value j. Thus, multiplying by $e_j^\top$ simply selects the j-th row of W_i.
For numerical variables, T_i is defined:

$$T_i = b_i + x_i \cdot W_i \qquad \in \mathbb{R}^D$$

where $b_i \in \mathbb{R}^D$ is the learned bias and $W_i \in \mathbb{R}^D$ is the learned weight vector. Thus, each numerical feature is embedded using a linear projection from $\mathbb{R} \to \mathbb{R}^D$.

Then, all tokenized features are stacked into:

$$T = \text{stack}[T_{cat_1}, T_{cat_2}, \ldots, T_{cat_m}, T_{num_1}, T_{num_2}, \ldots, T_{num_c}] \qquad \in \mathbb{R}^{n \times D}$$

A learnable [CLS] $\in \mathbb{R}^D$ token embedding is then appended to T, so that

$$T_0 = \text{stack}([cls], T) \qquad \in \mathbb{R}^{(n+1) \times D}$$

This sequence is passed through N Transformer layers $F_1, \ldots, F_N$, where $T_k = F_k(T_{k-1})$ for $k \in \{1, \ldots, N\}$. The final representation of the [CLS] token is used for prediction:

$$\hat{y} = \text{Linear}(\text{ReLU}(\text{LayerNorm}(T_N^{[CLS]})))$$

2.3 Numerical Feature Representations

Since the FT-Transformer embeds numerical features using a feature-specific linear projection into a D-dimensional space, the representation of a numerical variable prior to this projection can influence the resulting embedding and the model's ability to capture useful patterns.

Recent work by [4] showed that transforming numerical features before this projection can improve the performance of tabular deep learning models. Instead of directly using raw scalar values, numerical features can be converted into vector representations that better capture structure in the data.

One such approach is *Piecewise Linear Encoding* (PLE), which transforms a scalar numerical value into a vector representation based on discretization of the feature range. Each numerical feature is divided into T intervals defined by a sequence of bin boundaries, typically determined using empirical quantiles of the training data. A value x is then represented by a vector indicating its position relative to these boundaries:

$$\text{PLE}(x) \in \mathbb{R}^T.$$

The resulting vector is then projected through a linear layer to obtain the embedding used by the model.

3 Contributions and Innovations

The representation of numerical variables prior to the linear projection may substantially influence the resulting embeddings and the model's predictive performance. While previous work has explored alternative numerical embeddings, the effect of distribution-aware transformations prior to the projection layer has not been extensively examined in this context.

In this work, we investigate how different numerical feature representations affect the performance of the FT-Transformer across multiple tabular datasets. In particular, we introduce two transformations: simple Z-score normalization and a rank-based representation, and compare them to commonly used alternatives.

3.1 Numerical Feature Representations Evaluated

We evaluate four representations of numerical features applied before the feature-specific linear projection layer of the FT-Transformer:

- **Raw scalar input:** Numerical features are used in their original scalar form and directly passed to the linear projection layer.
- **Piecewise Linear Encoding (PLE):** As a reference method, we include the Piecewise Linear Encoding proposed in [4], which transforms numerical features into higher-dimensional vectors based on discretization into bins.
- **Z-score normalization:** As an initial alternative, we apply standard Z-score normalization to each numerical feature. While standardization is commonly used as a preprocessing step for numerical inputs in neural networks, we evaluate its effect here as the direct representation fed into the FT-Transformer embedding layer. For a feature x_i, let μ_i and σ_i denote the mean and standard deviation computed from the training data. The normalized value is defined as:
$$\tilde{x}_i = \frac{x_i - \mu_i}{\sigma_i}.$$
- **Rank-based representation:** Rank-based normalization maps each observation to its empirical percentile within the training distribution. Unlike Min–Max scaling, which relies only on the minimum and maximum values and is therefore sensitive to extreme observations, the rank transformation leverages the entire empirical distribution and is more robust to outliers. Compared to Z-score normalization, which standardizes the mean and variance while preserving the original magnitude scale, the rank transformation removes dependence on absolute scale and retains only the relative ordering of observations. As a result, the transformed values are distributed more uniformly over the input range of the linear embedding layer, which may facilitate more stable learning of feature embeddings. Let $\{x_i^{(1)}, \ldots, x_i^{(n)}\}$ denote the training values of feature i, and let k be the number of training samples with values less than or equal to x_i. The transformed value is defined as [6]:
$$r_i(x_i) = \frac{k - \frac{1}{2}}{n}.$$
This transformation maps each value to the interval $(0, 1)$ and represents its position within the empirical distribution of the feature.

All representations are applied prior to the feature-specific linear projection used by the FT-Transformer and are computed using statistics or reference data derived exclusively from the training set, ensuring that no information from the validation or test sets is used during preprocessing.

3.2 Experimental Evaluation Framework

To evaluate the effect of these representations, we conduct experiments across several tabular datasets. For each dataset, the FT-Transformer is trained using each of the four numerical feature representations described above. Model performance is evaluated using the ROC-AUC metric, which measures the ability of the model to rank positive instances above negative ones and is robust to class imbalance. This metric was also used in the reference works, aligning our work with prior studies.

Thereafter, we compare the resulting performance to that of a Random Forest classifier. This comparison allows us to assess whether improved numerical feature representations can narrow the performance gap between transformer-based models and tree-based methods.

3.3 Statistical Comparison Across Datasets

Performance comparisons on individual datasets may not reliably reflect general trends. To evaluate whether observed differences in evaluation metrics between representations are consistent across datasets, we first apply the Friedman test [7]. The comparison includes the FT-Transformer variants as well as the Random Forest baseline. The Friedman test is a non-parametric statistical test that compares the average ranks of the evaluated methods across all datasets and tests the null hypothesis

that all methods perform equivalently. If the Friedman test indicates a statistically significant difference between the methods, we then perform the Nemenyi post-hoc test to determine which specific pairs of methods differ.

4 Results

4.1 Datasets and Experimental Protocol

We evaluated the proposed representations on six tabular datasets. Five of these datasets were obtained from public repositories (Kaggle and the UCI Machine Learning Repository), while one dataset consists of an original clinical dataset of septic shock patients provided by Sheba Medical Center. The inclusion of this medical dataset introduces a real-world clinical prediction task characterized by different features and noisy measurements, reflecting complexities that may not always be present in benchmark datasets.

Across the datasets, we intentionally included varying dataset sizes as well as different compositions of numerical and categorical features in order to represent a range of tabular data complexities. Basic characteristics of the datasets are summarized in Table 1. Prior to modeling, duplicate rows were removed from all datasets to avoid bias from repeated observations.

Table 1: Summary of the datasets used in the experiments.

Dataset	Observations	Total Features	Categorical	Numerical
Heart Disease [8]	302	13	8	5
Stroke [9]	5,110	11	5	6
Septic Shock (medical)	5,579	28	3	25
Credit Card Default [10]	29,965	23	9	14
Bank Marketing [11]	41,176	19	10	9
Give Me Some Credit [12]	100,693	10	0	10

All datasets were processed using the same experimental pipeline. Each dataset was randomly split into 80% training data, 10% validation data, and 10% test data. Missing values were then imputed using statistics computed from the training set only in order to avoid data leakage: numerical features were imputed using the mean value, and categorical features were imputed using the most frequent category.

To maintain consistency across datasets, we did not perform dataset-specific hyperparameter tuning. Instead, all models were trained using a fixed configuration of architecture and training parameters across all experiments, allowing the comparison to focus on the effect of the numerical feature representations rather than dataset-dependent optimization choices. The full set of model and training hyperparameters used in the experiments is provided in Appendix A.2.

For each dataset and representation method, training was repeated across 10 random seeds (0–9), with each run trained for 40 epochs. Model performance was evaluated using the ROC-AUC metric on the test set. The reported score for each dataset-method pair corresponds to the average ROC-AUC across the 10 runs. In addition, we recorded the total training time required for each method across the seeds.

4.2 Performance Comparison

Table 2 presents the resulting performance comparisons across datasets and representations.

Table 2: Mean ROC-AUC across 10 seeds for each dataset and method. Within the FT-Transformer methods, the best result in each row is shown in bold. If the Random Forest result exceeds the best FT-Transformer result, it is underlined.

Dataset	FTT_RAW	FTT_ZSCORE	FTT_RANK	FTT_PLE	RF_RAW
Heart Disease	0.8538	**0.8576**	0.8437	0.8529	0.8370
Stroke	0.6797	0.7041	0.5459	**0.7056**	<u>0.7259</u>
Septic Shock (medical)	0.6669	0.6949	0.6876	**0.7207**	<u>0.7232</u>
Credit Card	0.7496	0.7576	0.7577	**0.7671**	0.7542
Bank Marketing	0.8013	0.8006	0.8005	**0.8022**	0.7718
Give Me Some Credit	**0.8563**	0.8494	0.7905	0.8540	0.8243

Overall, the FT-Transformer variants achieved competitive performance across the evaluated datasets, with Piecewise Linear Encoding (PLE) obtaining the best FT-Transformer result on several datasets, including Credit Card and Bank Marketing, although in some datasets the differences between representations were negligible. In contrast, the rank-based representation generally produced the lowest performance among the FT-Transformer methods. Random Forest achieved the best result on Stroke and Septic Shock and remained competitive on other datasets. Runtime comparisons (reported in Appendix A.3) show that the Random Forest baseline required roughly $\frac{1}{100}$ of the training time of the average FT-Transformer model, while still delivering competitive predictive performance.

4.3 Statistical Analysis

The Friedman test did not detect statistically significant differences between the evaluated methods ($\chi^2 = 6.27$, p-value $= 0.18$), indicating that no method consistently outperformed the others across the datasets. Therefore, the Nemenyi test was not applied.

5 Conclusions and future directions

In this work, we investigated how different representations of numerical features influence the performance of the FT-Transformer on tabular prediction tasks. We compared four representations applied prior to the feature-specific linear projection: raw scalar inputs, Piecewise Linear Encoding (PLE), Z-score normalization, and a rank-based transformation that we proposed.

The evaluation was conducted across five Kaggle/UCI datasets and a single medical dataset, with varying sizes and feature compositions. The results suggest that numerical feature representation may influence predictive performance, although the observed differences were generally small and varied across datasets. PLE often produced the strongest FT-Transformer results, while Z-score normalization achieved comparable performance in several datasets. The rank-based representation generally produced lower performance compared to the other methods. Overall, our proposed approaches did not outperform the existing alternatives, and the Friedman test did not detect statistically significant differences between the evaluated representations.

We also compared the FT-Transformer models to a Random Forest baseline. While the FT-Transformer variants achieved competitive predictive performance, Random Forest often matched or exceeded their accuracy while requiring substantially less training time.

Several limitations should be noted. Due to computational constraints, the experiments were conducted mainly on small to medium-sized datasets, which may limit the ability of deep models to fully capture complex feature interactions. In addition, hyperparameters were intentionally kept fixed across datasets to maintain comparability, although further tuning could potentially highlight strengths of certain representations.

Future work could explore new numerical representations tailored to transformer-based tabular models, such as encodings that capture distributional structure or monotonic patterns. Another promising direction is the development of embedding mechanisms that do not rely solely on linear projections, enabling richer transformations of numerical features before attention layers. Evaluating these approaches on larger datasets may also help reveal their full potential in modeling feature interactions.

References

[1] Ravid Shwartz-Ziv and Amitai Armon. Tabular data: Deep learning is not all you need. *Information Fusion*, 81:84–90, 2022.

[2] Liudmila Prokhorenkova, Gleb Gusev, Aleksandr Vorobev, Anna Veronika Dorogush, and Andrey Gulin. Catboost: unbiased boosting with categorical features. *Advances in neural information processing systems*, 31, 2018.

[3] Shriyank Somvanshi, Subasish Das, Syed Aaqib Javed, Gian Antariksa, and Ahmed Hossain. A survey on deep tabular learning. *arXiv preprint arXiv:2410.12034*, 2024.

[4] Yury Gorishniy, Ivan Rubachev, Valentin Khrulkov, and Artem Babenko. Revisiting deep learning models for tabular data. *Advances in neural information processing systems*, 34:18932–18943, 2021.

[5] Xin Huang, Ashish Khetan, Milan Cvitkovic, and Zohar Karnin. Tabtransformer: Tabular data modeling using contextual embeddings. *arXiv preprint arXiv:2012.06678*, 2020.

[6] C. Cunnane. Unbiased plotting positions — a review. *Journal of Hydrology*, 37(3):205–222, 1978.

[7] Janez Demšar. Statistical comparisons of classifiers over multiple data sets. *Journal of Machine learning research*, 7(Jan):1–30, 2006.

[8] johnsmith88. Heart disease dataset, 2021. `https://www.kaggle.com/datasets/johnsmith88/heart-disease-dataset`.

[9] Fedesoriano. Stroke prediction dataset, 2021. `https://www.kaggle.com/datasets/fedesoriano/stroke-prediction-dataset`.

[10] UCI Machine Learning Repository. Default of credit card clients dataset, 2010. `https://www.kaggle.com/datasets/uciml/default-of-credit-card-clients-dataset`.

[11] Mohit Janbandhu. Bank marketing dataset with social/economic context, 2021. `https://www.kaggle.com/datasets/mohitjanbandhu/bank-marketing-with-socialeconomic-context`.

[12] Kaggle. Give me some credit, 2011. `https://www.kaggle.com/c/GiveMeSomeCredit`.

A Appendix

A.1 TabTransformer architecture

Figure 2 presents the architecture of the TabTransformer. Each categorical value $x_i = j$ is embedded as $e_{\phi_i}(j) = [c_{\phi_i}, w_{\phi_{i,j}}]$, where c_{ϕ_i} is a column identifier enabling to distinguish the classes in column i from those in the other columns, and $w_{\phi_{i,j}}$ is the category-specific embedding from the lookup table.

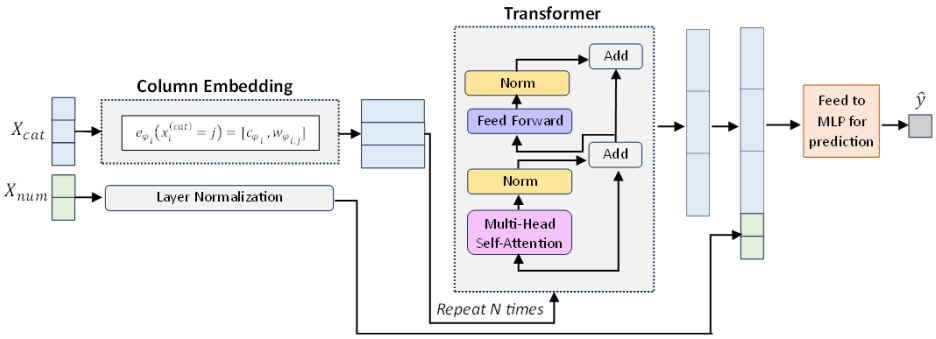

Figure 2: Architecture of the TabTransformer.

A.2 Hyperparameters

All model and training hyperparameters used in the experiments are summarized in Table 3.

Table 3: Hyperparameters used in all experiments. Default values were used across datasets to ensure consistent comparison between numerical representations.

Component	Hyperparameter	Value
FT-Transformer Architecture		
FT-Transformer	Embedding dimension (dim)	32
FT-Transformer	Number of transformer layers ($depth$)	3
FT-Transformer	Number of attention heads ($heads$)	8
FT-Transformer	Attention dropout	0.2
FT-Transformer	Feed-forward dropout	0.2
FT-Transformer	Output dimension	1 (binary classification)
Training Configuration		
Optimizer	AdamW	–
Training	Learning rate	10^{-4}
Training	Weight decay	10^{-4}
Training	Epochs	40
Training	Model selection criterion	Best validation ROC-AUC
PLE Representation		
PLE	Number of bins per feature	20
PLE	Bin fitting method	Quantiles (training set only)
Random Forest Baseline		
Random Forest	Number of trees ($n_estimators$)	100
Random Forest	Maximum depth (max_depth)	None
Random Forest	Minimum samples per leaf ($min_samples_leaf$)	1
Random Forest	Maximum features ($max_features$)	sqrt
Experimental Protocol		
Experiment	Random seeds	10 (0–9)
Experiment	Train/validation/test split	80/10/10
Imputation	Numerical features	Mean (training set)
Imputation	Categorical features	Most frequent (training set)

A.3 Training Time Comparison

Table 4 reports the total training time in minutes for each method across 10 seeds.

Table 4: Total training time (minutes) for each method across 10 seeds.

Dataset	**FTT_RAW**	**FTT_ZSCORE**	**FTT_RANK**	**FTT_PLE**	**RF_RAW**
Heart Disease	1.70	1.29	1.36	1.53	**0.11**
Stroke	10.87	10.91	10.77	10.59	**0.09**
Septic Shock (medical)	12.27	11.98	12.09	12.00	**0.14**
Credit Card	57.22	56.18	58.97	58.25	**0.87**
Bank Marketing	84.93	85.33	84.61	85.41	**0.53**
Give Me Some Credit	204.53	203.99	202.84	207.07	**2.13**